Project Report

1. Team members : Xiaokun Wang 999025521
Enhao Hu 999025208

Reused resources : handware.inc
readKeys function

2. How to play :

In Catrap, the player controls a character on a 12×12 grid. The goal is to eliminate all monsters and ghosts by attacking them from the side. The player can also dig through sand, push rocks, and climb stairs. All tiles are affected by gravity (except the ghosts and sand) – rocks and monsters fall down until they hit a solid tile or the floor.

Keys :

Directional pad (↑ ↓ ← →) : Move player
START (Enter) : Reset current level
A : Go to next level
B : Go to previous level

(Note: In the emulator used for testing, the keyboard mapping is as follows:
Keyboard A key → Game Boy B button (go to previous level)
Keyboard S key → Game Boy A button (go to next level)
This is why pressing A on the keyboard navigates to the previous level
and pressing S navigates to the next level during gameplay.)

Implemented features :

1.Title screen → any key starts level 0.
2.Level loading (up to 10 levels).
3.Player movement respecting tiles (walls, stairs, sand, rocks, enemies).
4.Rock pushing (only if empty space behind).
5.Enemy elimination (only horizontally, from the side).
6.Gravity for rocks and monsters (called after each move).
7.Stairs logic : player can move onto stairs at any time; moving up without stairs is only allowed if currently standing on stairs.
8.Win condition: wMonsterCount == 0 → next level.
9.After last level : ending screen → any key back to title.
10.A/B keys to jump between levels.
11.All VRAM/OAM updates happen strictly inside VBlank or while PPU is off.
12.Incremental background redraw – map changes are spread over several VBlanks (1 row per frame) to keep the load well below the VBlank time budget.
13.Tail-jump pattern for level transitions: UpdateGameState uses pop hl to discard the return address before jp StartLevel, avoiding a redundant ret/call pair and keeping the stack clean across state changes.
14.The process of the code is state machine:EntryPoint → TileScreen → StartLevel → MainLoop → EndingScreen → TileScreen.(Jump between the three state-function by "jp")

3. Implementation description :

Data Structure (WRAM)

name	purpose
wMapBuffer	12×12 logical tilemap
wCurrentLevel	Current level index
wPlayerX, wPlayerY	Player grid coordinates (0-11)
wMonsterCount	Remaining monsters + ghosts
wTargetX, wTargetY	Temporary move target
previous	ReadKeys internal state
current	ReadKeys rising-edge output
wMapDirty	Flag:1= redraw background needed
wRedrawRow	Next row to copy during incremental redraw
wFallingTile	Temporary storage during gravity passes

Algorithms

3.1 Level loading (LoadLevel)

Input: wCurrentLevel

Output: wMapBuffer filled, wPlayerX/Y, wMonsterCount

Steps:

1. HL = Levels + wCurrentLevel × MAP_SIZE (144). Because all levels are the same size, the address is computed by adding MAP_SIZE once per level index (ld de, MAP_SIZE; add hl, de loop), matching the pattern used in maze.asm.
2. Copy 144 bytes from HL to wMapBuffer
3. While copying, if tile == MONSTER or GHOST, increment wMonsterCount
4. Scan wMapBuffer to find TILE_PLAYER (7):

Compute index C = position

wPlayerY = C / 12 (by repeated subtraction), wPlayerX = C % 12

Replace that tile with TILE_NOTHING

3.2 Movement and interaction (TryMove)

Input: D = target X, E = target Y (player wants to move there)

Output: updates wMapBuffer, wPlayerX/Y, wMonsterCount, wMapDirty

Logic:

1. tile = GetTileAtXY(D, E)
2. If tile is NOTHING or STAIRS → doMove
3. If SAND → If in the horizontal direction → replace with NOTHING (UpdateOneTile), then doMove
4. If MONSTER or GHOST → kill only if E == wPlayerY (same row): replace with NOTHING, decrement wMonsterCount, then doMove
5. If ROCK → compute behind cell:

   behind_X = D + (D - wPlayerX), behind_Y = E + (E - wPlayerY)

if behind cell is NOTHING: move rock there (update both cells), then apply gravity (no player move yet)

6. If none of above → blocked (return)

doMove: wPlayerX = D, wPlayerY = E, then call ApplyGravityToAll and ApplyGravityToPlayer.

UpdateOneTile(A, D, E): writes A to wMapBuffer[D+E*12] and sets wMapDirty = 1.

3.3 Gravity (ApplyGravityToAll and ApplyGravityToPlayer)

ApplyGravityToPlayer (called after each move):

1. If the player is standing on STAIRS, do nothing (ladder stops falling).
2. While the tile below player is NOTHING AND player is not standing on STAIRS:

wPlayerY++ (move down one row)

3. Stops when blocked or on ladder.

ApplyGravityToAll (called after each move):

Bottom-up scan: for rows MAP_ROWS-2 down to 0, for each column:

If tile at (col, row) is ROCK, MONSTER:

Check tile directly below (col, row+1)

If below is NOTHING and not the player's position:

Swap tile down (clear current, write tile below)

Set moved flag

Repeat whole scan until no tile moves in a full pass.

3.4 Incremental background updates

To guarantee that VRAM writes never exceed the VBlank budget, the game does not redraw the full 12×12 map in one frame. Instead:

UpdateOneTile sets wMapDirty = 1 whenever wMapBuffer changes.

In MainLoop, when wMapDirty is seen, wRedrawRow is reset to 0, starting an incremental redraw.

During each VBlank, DrawMapRows copies only one row (12 tiles) from wMapBuffer to the background tilemap.

After each VBlank, wRedrawRow advances. The redraw finishes after 12 frames.

This spreads the VRAM write over multiple frames, keeping the per-frame load low.

UpdateOAM (called every VBlank):

Translates (wPlayerX, wPlayerY) to hardware OAM pixel coordinates and writes 4 bytes directly to STARTOF(OAM) entry 0. No ShadowOAM is used; because the game has only one sprite object, a direct write during VBlank is sufficient.

OAM_Y = (wPlayerY + MAP_DISPLAY_ROW) * 8 + 16

OAM_X = (wPlayerX + MAP_DISPLAY_COL) * 8 + 8

DrawMapToBackground (called with PPU off, e.g. from StartLevel, for the initial full draw) and DrawMapRows (called inside VBlank, ROWS_PER_FRAME = 1 row per frame):

1. DrawMapToBackground calls DrawMapRows for all 12 rows at once (safe only with PPU off). During gameplay, MainLoop sets wRedrawRow = 0 when wMapDirty fires, then each VBlank DrawMapRows copies ROWS_PER_FRAME (= 1) row and advances wRedrawRow; the full 12-row redraw spreads over 12 frames.

2. After each row, skips 20 unused columns (32−12) to reach next display row.

3. Does not touch OAM.

4. wMapDirty is cleared by MainLoop when it schedules the redraw (sets wRedrawRow = 0); DrawMapRows itself does not touch wMapDirty.

3.5 Main loop timing control

1. All VRAM/OAM writes are inside WaitVBlank (or after SafeTurnOffLCD).

2. wMapDirty triggers a multi-frame redraw: MainLoop clears wMapDirty and resets wRedrawRow to 0; each subsequent VBlank, DrawMapRows copies ROWS_PER_FRAME rows and increments wRedrawRow until it reaches MAP_ROWS (redraw idle).

3. UpdateOAM writes the player sprite directly to hardware OAM during VBlank.

4. These two guarantees prevent graphical corruption.

4. Shortcomings of the implementation :

1. No smooth transitions.

2. A/B keys skip win condition : pressing A or B immediately jumps to another level without checking wMonsterCount.

3. Currently, by setting ROWS PER FRAME = 1, 12 rows are divided into 12 frames for line - by - line rendering. After the player moves, it takes 12 frames for the map to be completely refreshed. This may cause a slight visual lag.

5. WARM usage :

WRAM is used for game state, map buffer, shadow OAM and temporary variables. All variables are placed in the WRAM0 section.

Variables	Bytes	Description
wCurrentLevel	1	Level index
wMapBuffer	144	12×12 tile map
wPlayerX , wPlayerY	2	Player position
wMonsterCount	1	Enemy counter
wTargetX wTargetY	2	Move target
previous	1	ReadKeys internal state
current	1	ReadKeys rising-edge output
wMapDirty	1	Redraw flag
wRedrawRow	1	Next row to redraw
wFallingTile	1	Temporary gravity storage

Total 155 bytes

6. Cycles calculations of VRAM/OAM :

The Game Boy's PPU can only safely access VRAM and OAM during VBlank (rLY = 144–153). This window lasts 10 × 456 = 4560 CPU cycles. MainLoop performs VRAM and OAM updates here.

1. DrawMapRows (1 row per VBlank, ROWS_PER_FRAME = 1)

Setup – address computation (runs once per call):

push bc (16) + compute DE via shifts/adds (4+4+4+4+4+4+8+4+8+8+4=56) + compute HL via add hl,hl x5 + ld bc + add hl,bc (8x5+12+8=60) + pop bc + ld b,c (12+4=16)

Setup total: 16 + 56 + 60 + 16 =148

Inner loop (.colLoop, 12 tiles):

Each tile: ld a,[de] (8) + ld [hl+],a (8) + inc de (8) + dec c (4) + jr nz (12/8)

Columns 1–11 (jr taken): 40 × 11 = 440 cycles

Column 12 (jr not taken): 8+8+8+4+8 = 36 cycles

Inner loop total: 440 + 36 = 476 cycles

Row advance (skip 20 unused tilemap columns, worst-case carry):

ld a,l (4) + add 20 (8) + ld l,a (4) + jr nc taken (12) + inc h (4) = 32 cycles

Outer loop tail (1 row, jr not taken) + ret:

dec b (4) + jr nz not taken (8) + ret (16) = 28 cycles

DrawMapRows summary:

Setup 148 + Inner loop 476 + Row advance 32 + Tail+ret 28 = 684 cycles (function body)

+ call DrawMapRows: 24 cycles → **Total: 708 cycles**

2. UpdateOAM (every VBlank)

UpdateOAM writes the player sprite directly to hardware OAM entry 0 (STARTOF(OAM)), bypassing ShadowOAM entirely:

ld hl, STARTOF(OAM) (12) + ld a , [wPlayer_Y] (16) + add MAP_DISPLAY_ROW + add a×3 + add 16 (28) + ld [hl+] a(OAMY) (8) + ld a , [wPlayer_X] (16) + add MAP_DISPLAY_COL + add a×3 + add 8 (28) + ld [hl+] a(OAMX) (8) + ld [hl], TILE_PLAYER (12) + inc hl (8) + ld [hl], 0 (12) + ret (16) = 164 cycles

Total: call UpdateOAM (24) + 164 = 188 cycles

3. VBlankUpdate overhead (worst case: wMapDirty = 1, draw frame)

Dirty path (clears wMapDirty, resets wRedrawRow):

ld a,[wMapDirty] (16) + and a (4) + jr z not taken (8) + xor a (4) + ld [wMapDirty],a (16) + ld [wRedrawRow],a (16) = **64 cycles**

Redraw dispatch (triggers DrawMapRows call):

ld a,[wRedrawRow] (16) + cp MAP_ROWS (8) + jr nc not taken (8) + ld b,a (4) + ld c,ROWS_PER_FRAME (8) + call DrawMapRows (24) + ld a,[wRedrawRow] (16) + add (8) + ld [wRedrawRow],a (16) = **108 cycles**

ret: 16 cycles call UpdateOAM: 24 cycles

VBlankUpdate overhead total: 64 + 108 + 16 + 24 = 212 cycles (excl. DrawMapRows body and UpdateOAM body)

4. Total worst-case VBlank load

DrawMapRows body (1 row):	684 cycles
UpdateOAM body:	164 cycles
VBlankUpdate overhead:	212 cycles
call VBlankUpdate	24 cycles

Total: 1084 cycles (budget: 4560 cycles, margin: 3476 cycles (76%))

Removing ShadowOAM and CopyShadowOAMtoOAM (which alone cost 3908 cycles) and replacing them with a direct 4-byte UpdateOAM (188 cycles total) reduces the worst-case VBlank load from 4764 cycles (over budget) to 1084 cycles (well within the 4560-cycle budget, 76% margin).

Non-draw frames (wRedrawRow ≥ MAP_ROWS): VBlankUpdate overhead shrinks to ~80 cycles (dirty check + skip-draw branch + UpdateOAM call). Total ≈ 268 cycles.

7. Total CPU use outside VBlank (per frame, minimal)

call readKeys : 24 + 284 = 308 cycles

call UpdateGameState : 24 + 228 = 252 cycles (minimal, no input)

Sum = 308 + 252 = 560 cycles

This is the CPU time spent on game logic each frame.

(Additional Note)Tile creation tool :

Tiles were created using Game Boy Tile Designer (GBTD) (https://www.devrs.com/gb/hmgd/gbtd.html), a standard tool in the Game Boy homebrew community for designing 8×8 tiles. GBTD provides a pixel-grid interface and exports the tile data as byte patterns, which were then copied directly into the assembly source.

Each byte represents one row of 8 pixels; CopyTilesToVRAM doubles each byte into the 2bpp VRAM format required by the Game Boy PPU. The tiles used are:

```
db $00,$00,$00,$00,$00,$00,$00,$00 ; Tile 0: NOTHING
db $00,$FF,$00,$FF,$00,$FF,$00,$FF ; Tile 1: STAIRS
db $3C,$42,$5A,$42,$42,$5A,$42,$3C ; Tile 2: MONSTER
db $3C,$42,$81,$81,$81,$81,$42,$3C ; Tile 3: ROCK
db $AA,$55,$AA,$55,$AA,$55,$AA,$55 ; Tile 4: SAND
db $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF ; Tile 5: WALL
db $3C,$7E,$DB,$FF,$FF,$FF,$A5,$A5 ; Tile 6: GHOST
db $81,$42,$3C,$5A,$7E,$42,$42,$3C ; Tile 7: PLAYER
```